EE8350 - AES Decode

Joel Kuehne <kuehn348@umn.edu>

This report details the verification plan and preliminary test framework for the FPGA AES Decryption Core from OpenCores. This core contains an implementation of the AES decryption algorithm described in FIPS-197 and is parameterizable for key lengths of 128, 192, and 256 bits. The core is verified against a behavioral reference model of the AES core and against selected test vectors from FIPS-197, SP-800a, and AESAVS specifications. Primary coverage is obtained from system-level tests, additional coverage can be gained from module level tests targeting the Decryption Engine and Key Expander individually as needed.

Introduction

The Advanced Encryption Standard (AES) is a symmetric key encryption/dectyption standard based on the Rijndael block cipher algorithm. The OpenCores AES Decryption Core for FPGA implements the decryption portion of this algorithm. Described in "AES Dectyption Core for FPGA: Specification", it supports compile time configurations for 128, 192, and 256-bit decryption. It is deigned for high throughput on FPGA targets and can achieve a peak throughput of 3Gbps for 256-bit keys while only requiring around 2000 LUTs. It is a fully synchronous design with one clock domain.

The core is split up into three components. The Key Expander accepts key text on the "kt" interface and expands it into a series of round keys. The Decryption Engine accepts input cipher text on the "ct" interface and decrypts it into plain text which is returned on the "pt" interface. The Key Expander and Decryption Engine communicate through the Key Schedule Buffer, which implements a Last-In-First-Out (LIFO) buffer that holds the different round keys.

Top Module

OpenCores provides three different top modules for their AES core, one for each supported keylength. The three different parameterizations are identical apart from the fact that they require different key expansion models and that they use different numbers of rounds as specified in the AES standard (10, 12, and 14 respectively for the 128, 192, and 256-bit configurations). The top level module is connected directly to the testbench via the "pt", "ct", and "kt" interfaces as described in Table 1.

Decryption Engine

The decryption engine implements the four inverse operations required to decrypt the provided ciphertext. The inverses of AddRoundKey, MixColumns, ShiftRows, and SubBytes are implemented in InvAddRoundKey, InvMixColumns, InvShiftRows, and InvSubBytes

respectively. InvAddRoundKey reverses the Galois field addition by XORing the ciphertext with the roundkey. InvMixColumns reverses the column-mixing matrix operation by multiplying by the inverse matrix in the Galois field. InvShiftRows implements the inverse of the row shift operation by repacking bytes. InvSubBytes computes the byte substitution via a precomputed 8-input lookup table.

The engine accepts input ciphertext on the the “ct” interface and stores it in the State register. One decryption round is performed every cycle. Taking into account the additional cycle of latency to assign the State register, each 128-bit decryption takes ROUND_COUNT + 1 cycles (11, 13, and 15 for 128, 192, and 256-bit key length configurations respectively). The testbench drives ciphertext to the decryption engine from the “ct” interface, observes the primary output on the “pt” interface, observes the round key input for scoreboarding, and observes the input to the InvSubBytes and InvMixColumns modules for coverage tracking.

Key Expander

The core implements three different key expansion modules, one for each supported key length. Each module implements the rotation, word substitution, and round constant addition as specified in FIPS-197. The substitution is implemented with a precomputed 8-input lookup table. The result of the key expandsion is sent to the key schedule buffer. Similar to the Decryption Engine, the Key Expander requires one cycle per round key plus an additional cycle of latency to populate the round key register (11, 13, and 15 cycles for 128, 192, and 256-bit key length configurations). The testbench drives key text to the expander on the “kt” interface and observes the expanded key output immediately before the Key Schedule Buffer.

Signal		Source	Width	Description
Ports	**clk**	TB	1	Core clock
	rst	TB	1	Active high synchronous reset
	kt	TB	128/192/256	Key input
	kt_vld	TB	1	Key input valid signal
	kt_rdy	DUT	1	Key input ready signal
	ct	TB	128	Ciphertext input
	ct_vld	TB	1	Ciphertext input valid signal
	ct_rdy	DUT	1	Ciphertext input ready signal
	pt	DUT	128	Plaintext output
	pt_vld	DUT	1	Plaintext output valid siganl
Decrypt	**rkey_decrypt**	DUT	128	Decryptor key schedule buffer data
	rkey_vld_decrypt	DUT	1	Decryptor key schedule buffer valid
	next_rkey_decrypt	DUT	1	Decryptor key schedule request
	inv_shiftrows_out	DUT	128	Input to InvSubBytes module
	inv_addrkey_out	DUT	128	Input to InvMixColumns module
Expand	**rkey_keyexp**	DUT	128	Expander key schedule buffer data
	rkey_vld_keyexp	DUT	1	Expander key schedule buffer valid
	rot_word_out	DUT	128	Input to word substitution LUT

Table 1: DUT connections for the AES testbench.

Verification

The primary goal of the verification of the key expansion module is to verify that the core correctly implements the AES decryption algorithm described in FIPS-197. Apart from this, the core must correctly implement ready/valid interfaces and must respond properly to reset assersion. Primary coverage is driven through constrained random verification at the top-level. Additonal directed testing using known test vectors from the FIPS-197, SP-800a, and AESAVS specifications. These vectors were provided by OpenCore with the original repository source. The complete verification plan is in below.

ID	Requirement	Description
	Verification	
1	**Configuration**	
1.1	128/192/256-bit Key Length	Verify that the DUT supports compile time configuration for 128, 192, and 256-bit key lengths.
	Run each test on all three DUT configurations.	
2	**Reset**	
2.1	Synchronous Warm Reset	Verify that DUT state transitions to 0 on reset and any stored key or ciphertext is dropped.
	Assert that both the expander and decoder modules return to state 0 after reset.	
	Verify that both transactions before and after reset are processed successfully.	
	Cover all FSM states at the moment of reset assertion via constrained random testing.	
3	**Decryption Engine**	
3.1	Plaintext X-Propgatation	Verify that output signals on the “pt” interface are known when “pt_vld” is asserted.
	Assert that “pt” is known when “pt_valid” is asserted.	
3.2	Decryption Correctness	Verify that the Decryption Engine correctly implements the AES decryption algorithm.
	Verify that the plaintext output is consistent with that of the behavioral model.	

Cover 64 evenly distributed bins for ciphertext and keytext through CRV.		
InvSubBytes Correctness	Verify that the InvSubBytes module correctly implements the inverse SBox transform through CRV.	3.3
Cover all 256 entries in the InvSbox LUT for all byte lanes.		
InvMixColumns Correctness	Verify that the InvMixColumns module correctly implements the inverse column mixing operation.	3.4
Cover 64 evenly distributed bins for in InvMixColumns through CRV.		**4**
Key Schedule Buffer		
Round Key X-Propagation	Verify that output signals on the “rkey_out” interface are known when “rkey_vld_out” is asserted.	4.1
Assert that “rkey_out” is known when		4.2

“rkey_vld_out” is asserted.		
Round Key LIFO Ordering	Verify that round keys are returned on the “rkey_out” interface in the opposite order they are provided on the “rkey_in” interface.	
Assert LIFO ordering between the “rkey_in” and “rkey_out” interfaces.		**5**
Key Expander		
Round Key X-Propagation	Verify that output signals on the “rkey_in” interface are known when “rkey_vld_in” is asserted.	4.1
Assert that “rkey_in” is known when “rkey_vld_in” is asserted.		
Key Expander Correctness	Verify that the Key Expander correctly implements the AES key expansion algorithm.	4.1

Assert that “rkey_in” is known when “rkey_vld_in” is asserted.	

Top

Decryption Engine

Key Expander

Key Schedule Buffer

Preliminary Testbench Components

Conclusions